How ELIXOR Works

The database, the backend, the frontend, security and tooling, explained step by step

Project: ELIXOR (github.com/Zageraus/Elixo) · Course: Database Systems, BCSE307L · Database: PostgreSQL 17 on Supabase · 28 October 2026

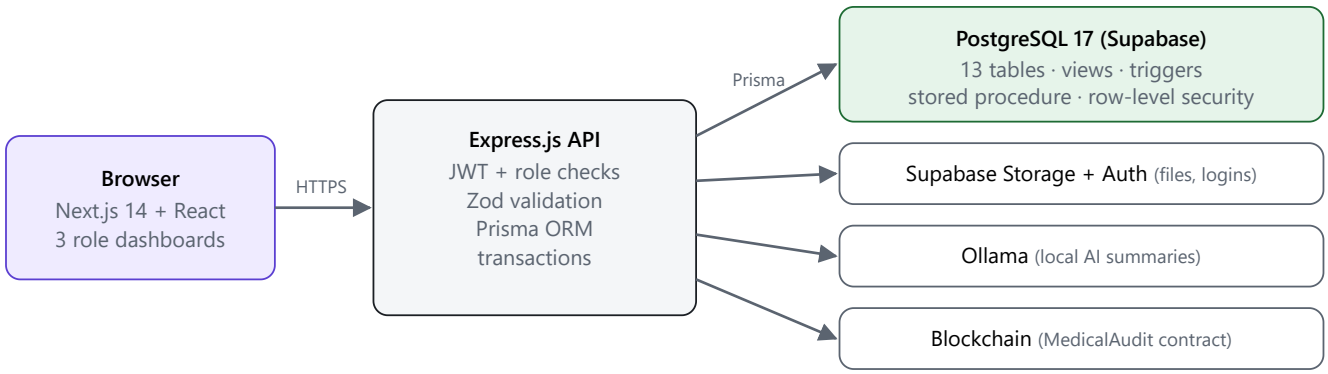
In one paragraph. ELIXOR is a clinic management system. Patients book appointments with doctors, pay the consultation fee, keep a health profile and upload lab reports; doctors accept visits and write prescriptions; administrators watch platform figures and manage accounts. All of it is stored in one PostgreSQL database designed to third normal form, with keys, constraints, triggers, views and a stored procedure enforcing the rules. A Next.js website talks to an Express.js API, and the API talks to the database only through the Prisma ORM, wrapping every multi-step change in a transaction. On top of that, report and prescription fingerprints can be anchored on a blockchain so any later tampering is detectable, and a local AI model can summarise lab reports.

1. Who uses it and what they can do

Role	What they can do
Patient	Register and log in · keep a health profile (allergies, blood group, date of birth, emergency contact) · find doctors by specialization and request an appointment · pay the fee once the doctor accepts (simulated checkout) · view prescriptions and reports page by page · upload a report and get an AI summary · open uploaded PDFs through short-lived links · verify a record has not been tampered with
Doctor	Register with specialization, licence number and fee · see their appointment queue with each patient's allergies and payment status · accept, decline or complete visits · write prescriptions with several medicines (warned about the patient's allergies)
Admin	See 12 live platform figures and a weekly doctor-load chart · search, filter and page through patients and doctors · deactivate, reactivate or permanently delete accounts (affected patients are notified)

Everyone uses the same login page. After login, each person only ever sees their own role's dashboard.

2. The big picture



Only the API touches the database. The browser never holds a database key.

Layer	Where it lives	Job
Frontend	`frontend/` (Next.js 14, Tailwind)	Login page, the three dashboards, forms with validation, search, filters and pagers

Backend API	`backend/src/routes/`	One file per area: auth, appointments, payments, prescriptions, reports, patients, notifications, admin
ORM	`backend/prisma/schema.prisma`, `src/config/prisma.ts`	Maps every table to a typed model; all reads and writes go through it
Database	`db/migrations/001–012`	Tables, constraints, indexes, triggers, views, the stored procedure, row-level security
File storage and passwords	Supabase Storage and Supabase Auth	Private bucket for report PDFs; bcrypt-hashed email passwords
AI and blockchain	`src/services/`, `contracts/`	Report summaries from a local model; tamper-evident fingerprints on a chain

3. The database

3.1 Thirteen tables, one Users table

Group	Tables
People	`users` (every patient, doctor and admin, told apart by a `role` column) · `doctor_profiles` · `patient_profiles` (1:1 extensions) · `auth_nonces` (wallet sign-in challenges)
Clinical work	`appointments` · `payments` · `prescriptions` · `prescription_items` · `medical_reports` · `ai_analyses`
Integrity and history	`blockchain_transactions` · `audit_logs` · `notifications`

The brief warns against separate Admin, Doctor and Patient tables. ELIXOR keeps everyone in `users`; attributes that only one role has (a doctor's licence and fee, a patient's allergies) live in the matching profile table, keyed by the same user id. So `users` has no columns that are empty for whole groups of people.

3.2 Relationships

Relationship	Cardinality	How it is stored
A patient **books** appointments; a doctor **attends** them	1 : N each	`appointments.patient_id`, `doctor_id` → `users`
An appointment is **paid by** at most one payment	1 : 0..1	`payments.appointment_id`, UNIQUE
A doctor **issues** prescriptions to a patient, optionally for a visit	1 : N	`prescriptions.doctor_id`, `patient_id`, `appointment_id`
A prescription **contains** medicines	1 : N (weak entity)	`prescription_items(prescription_id, line_no)`
A report **belongs to** a patient and is **analysed by** the AI	1 : N	`medical_reports.patient_id`; `ai_analyses.report_id`
A report or prescription is **anchored by** blockchain transactions	1 : N	`blockchain_transactions`, CHECK: exactly one parent

The full ER diagram, relational schema and data dictionary are in `docs/DATABASE_DESIGN.md`.

3.3 Normalization: what changed and why

Every table has a primary key, and every non-key column depends on the whole key, so the schema is in 2NF. Two places broke the higher forms, and migration 012 fixed them:

Before	Problem	After
`prescriptions.medications` held a JSON list of medicines	Repeating group (not 1NF)	Each medicine is a row in `prescription_items`, numbered by `line_no`
`payments` stored `patient_id` and `doctor_id`	Transitive dependency (not 3NF): payment → appointment → patient	Columns dropped; the patient and doctor are reached through the appointment

Kept on purpose: `prescriptions` still stores `patient_id` and `doctor_id`, because a prescription can exist without an appointment, so the appointment does not determine them. A trigger makes sure they match whenever an appointment is linked. `payments.amount` is a snapshot of the fee at the moment of payment, not a copy of the doctor's current fee.

3.4 Rules the database enforces by itself

Kind	Examples
Keys	13 primary keys, 17 foreign keys with ON DELETE CASCADE or SET NULL chosen per relationship
UNIQUE	email, wallet address, licence number, one payment per appointment, receipt number, (prescription, line number)
CHECK	blood group from a fixed list · fee ≥ 0 · amount > 0 · nobody books or prescribes for themselves · a refunded payment always has a refund time · duration > 0 · file hash is 64 hex characters
NOT NULL / DEFAULT	required fields; ids, timestamps and statuses fill themselves in
ENUM types	user role, appointment status, report type, payment method and status, notification type, chain status

3.5 Triggers, views and the stored procedure

Object	What it does
Triggers (13)	Keep `updated_at` current · write every change to reports, prescriptions, appointments, payments and roles into `audit_logs` · notify the doctor of a new request and the patient of each status change, report and prescription · check that an appointment's doctor is a doctor and patient is a patient · refund a paid appointment when it is cancelled · keep a prescription consistent with its appointment
Views (5)	`v_patient_dashboard`, `v_doctor_dashboard` (joins for the appointment lists) · `v_report_with_analysis` (each report with its latest AI summary and anchor, via LATERAL joins) · `v_admin_analytics` (COUNT, SUM and AVG across the platform) · `v_doctor_weekly_load` (GROUP BY doctor and week)
Stored procedure	`sp_create_prescription(doctor, patient, appointment, notes, items, INOUT id)` checks the input, inserts the prescription and one row per medicine, and returns the new id
Indexes (~30)	Every foreign key and common filter or sort column · `pg_trgm` indexes so the admin name and email search can use an index · a vector index for AI similarity search

4. One request from start to finish: a doctor issues a prescription

1. The doctor picks a patient. The page shows the patient's allergies in red before any medicine is typed.
2. The browser sends `POST /api/prescriptions` with the patient, optional appointment, medicines and notes, plus the login token in the `Authorization` header.

3. `requireAuth` checks the token's signature and expiry and confirms in the database that the account is still active. `requireRole("doctor","admin")` rejects anyone else with 403.
4. Zod checks the body: valid ids, 1 to 30 medicines, each with a name, dosage, frequency and a positive number of days. Bad input gets 400 before the database is touched.
5. `withAuditUser()` opens a **transaction** and records the doctor as the acting user for the audit trigger.
6. Inside it, Prisma calls the stored procedure: `CALL sp_create_prescription(...)`. The procedure inserts the prescription header and every medicine row.
7. Triggers fire inside the same transaction: the consistency check, the audit-log entry (with the doctor's id) and the patient's notification.
8. If anything fails, for example a medicine with 0 days, the whole transaction rolls back and nothing is saved. Otherwise it commits.
9. If anchoring is on, the backend fingerprints the medicines and writes the fingerprint to the blockchain.
10. The API returns the new prescription; the page refreshes the first page of the doctor's prescription history.

5. Transactions

A transaction makes several changes behave as one: they all commit, or none do (atomicity), the database stays valid (consistency), concurrent users do not see each other's half-done work (isolation), and committed data survives a crash (durability). ELIXOR uses `prisma.$transaction` for every change that touches more than one row:

Operation	Steps that commit or roll back together	Why it matters
Doctor registration	Insert user · insert doctor profile	No doctor account without a licence and fee
Paying for a visit	Lock the appointment row (`SELECT … FOR UPDATE`) · check its status and that it is unpaid · insert the payment	Two clicks at once cannot pay twice; tested with simultaneous requests
Issuing a prescription	Call the procedure (header and medicines) · read it back	No prescription with missing medicines
Deactivating a user	Notify affected patients · deactivate the account · cancel upcoming visits (the refund trigger runs inside)	Patients are never left with a cancelled, paid visit and no refund
Deleting a doctor	Notify patients · delete the account (cascades)	Notifications are not lost if the delete fails
Changing appointment status	Check the doctor owns it · update it (refund and notification triggers)	A doctor cannot change someone else's appointment

6. Security

Concern	How ELIXOR handles it
Passwords	Email passwords are stored by Supabase Auth as **bcrypt** hashes. The backend never stores or logs a password.
Login and logout	On login the backend issues a signed **JWT** (12-hour expiry) holding the user id and role. Logout deletes it from the browser; an expired or rejected token signs the user out automatically.
Who is logged in	`GET /api/auth/me` returns the current user from the database, so a changed role or a deactivated account takes effect immediately.

Role-based pages and menus	`RoleGate` asks `/me` before showing a dashboard. A patient who edits their stored role to "admin" is still sent back to the patient dashboard. Each dashboard shows only its role's menu and actions.
Role-based API access	`requireRole` on every protected route, and every query filtered to the signed-in user (a patient asking for another patient's prescription gets 404).
Database-level access	Row-level security policies on every table restrict rows per user for any client using a user token; they act as a second layer behind the API.
SQL injection	Prisma sends every value as a bind parameter. The few raw queries (views, the procedure) use Prisma's tagged templates, which also bind values.
Secrets	Database URL, JWT secret and keys live in `.env` files read with dotenv. All `.env` variants and database backups are git-ignored; the repository only has `.env.example` templates.

7. Application features

Feature	Where to see it
Search	Admin → User Management: search by name, email or specialization (server-side, case-insensitive)
Filtering	Admin: patients vs doctors tabs and Active / Deactivated filter · Patient: doctors by specialization
Pagination	Admin user list (8 per page) · patient and doctor prescriptions and reports (5 per page). The API returns `items`, `total`, `page` and `total_pages`.
Form validation	In the browser (required fields, date limits, length limits) and again on the server with Zod
Exception handling	One error handler turns problems into clear responses: 400 invalid input or a broken database rule, 401 not logged in, 403 wrong role, 404 not found, 409 duplicate or conflict, 503 database unreachable
Responsive UI	Tailwind layouts that stack into one column on phones

8. AI summaries and blockchain anchoring

AI. When a patient uploads a report with its text, the backend sends the text to a local Ollama model. The model returns a summary, flagged findings, a severity and a confidence score, which are checked and saved in `ai_analyses`. The PDF itself never goes to the model, and no data leaves the machine.

Blockchain. Each report file and each prescription's medicines get a SHA-256 fingerprint, which is written to the `MedicalAudit` smart contract. **Verify integrity** re-computes the fingerprint and compares it with the chain. Moving medicines into `prescription_items` did not change any fingerprint: all 10 anchored prescriptions still verify. The details are in `docs/ELIXOR_Blockchain_Workings.pdf`.

9. Running, testing and shipping

Task	How
Run locally	`npm --prefix backend run dev` (port 4000) and `npm --prefix frontend run dev` (port 3000), with `DATABASE_URL` and the Supabase keys in `backend/.env`
Set up a new database	Run `db/ALL_IN_ONE_SETUP.sql` in the Supabase SQL editor (all migrations, then seed data)
Automated checks	GitHub Actions on every push: Prisma schema validation, typecheck and build of both apps, smart-contract tests, Docker image builds. All passing.

Containers	A Dockerfile per app and `docker-compose.yml`; CI can push the images to Docker Hub
Backup and restore	`scripts/db-backup.sh` (pg_dump, custom format) and `scripts/db-restore.sh` (pg_restore in one transaction)
Deployment	Database on Supabase, backend on Render (`render.yaml`), frontend on Vercel
Version control	GitHub, `main` and `dev` branches, 36 commits with descriptive messages

The full API was tested end to end against the live database: 35 checks across all three roles, including the transaction, procedure, trigger and access-control cases. Test data was removed afterwards.

10. Where each rubric requirement is met

Requirement	Evidence	Status
Proposal and Gantt chart	`docs/PROJECT_PROPOSAL.md`	team names
ER diagram, schema, data dictionary	`docs/DATABASE_DESIGN.md` §1–4	done
≥ 4 entities, ≥ 3 relationships, ≥ 8 tables	5 primary entities, 14 relationships, 13 tables	done
Keys, constraints, 3NF	§3.3–3.4; migration 012	done
Indexes	§3.5; `DATABASE_DESIGN.md` §7	done
CRUD, joins, aggregates, views	All screens; the five views	done
Transactions, stored procedure, trigger	§4–5; `sp_create_prescription`; 13 triggers	done
ORM	Prisma for every table query	done
Login, dashboard, search, filter, pagination, validation, errors, responsive	§7	done
JWT, logout, user info, 2+ roles, role-based menus, pages and API	§6	done
Password hashing, SQL-injection safety, secrets in .env	§6	done
GitHub: branches, commits, .gitignore, README	`main` + `dev`, 36 commits	make private
Backup and restore	`scripts/`	run once
Docker, Docker Hub	Dockerfiles; images build in CI	push images
Cloud deployment, live URL	Render + Vercel config	deploy
Bonus: CI/CD, best practice	GitHub Actions; managed database, audit logging, automated tests	done

11. Questions an examiner may ask

Why are all users in one table? Roles change and share most attributes (name, email, login). One `users` table with a `role` column avoids duplicating people across tables; role-only attributes sit in 1:1 profile tables, so nothing is left empty.

Where is the ORM, and why is there still some SQL? All table reads and writes use Prisma. Views and the stored procedure have no Prisma builder, so they are called through Prisma's parameterised raw queries, which the brief allows for advanced database features.

Where is bcrypt? Supabase Auth stores email passwords as bcrypt hashes; the backend only passes the password to it during login and registration.

Show a transaction rolling back. Issue a prescription where one medicine has 0 days. The CHECK constraint on `prescription_items` fails, the procedure call raises an error and the prescription header is rolled back too: no row is left behind.

Is `prescriptions` really 3NF if it stores the patient and the appointment? Yes. `appointment_id` is optional, so it does not determine the patient in general. When it is present, a trigger guarantees consistency.

What stops two payments for one visit? The checkout transaction locks the appointment row with `FOR UPDATE`, and `payments.appointment_id` is UNIQUE as a final guard.

Why use the database for rules like refunds? A trigger runs no matter which code path changes the data, so the rule cannot be skipped by a bug in one route.

12. Known limitations

- Payments are simulated; no real gateway is called and no card data is collected.
- The AI needs the report's text pasted in; there is no OCR of scanned PDFs yet.
- Anyone can register as a doctor; there is no licence verification workflow.
- Blockchain anchoring runs on a local chain; the contract is ready for the Sepolia testnet but not deployed there.
- The login token is kept in the browser's local storage, which is simple but readable by injected scripts; an httpOnly cookie would be stronger.